
Architecture Design Document (ADD)

MECHRABOT: Mechanical Maintenance Smart Assistant

Project Blueprint & Technical Roadmap

1. Executive Summary

The goal of this project is to build an advanced Retrieval-Augmented Generation (RAG) system capable of parsing and comprehending professional English mechanical maintenance manuals. The system is specifically designed to tackle strict engineering challenges, such as retrieving highly precise "Torque Values" without hallucination. It will process technician queries in Arabic, internally rewrite them for optimal vector search, and deliver highly structured, professional responses. The architecture supports a flexible, hybrid deployment (Local/Cloud) to ensure absolute data security while maximizing available hardware efficiency.

2. Tech Stack & Tooling

The tooling has been carefully selected to strike the perfect balance between retrieval accuracy, latency, and architectural control.

The Stack

- **Core Orchestrator:** Haystack 2.x (for building explicit, highly controllable data pipelines).
- **Document Parsing:** Docling (for reading complex tables and executing Header-based Chunking to preserve document structure).
- **Vector Database:** Qdrant (hosted locally via Docker to support Hybrid Search capabilities).
- **Embedding Model:** BAAI/bge-m3 (Open-source, cross-lingual capabilities, outputting both Dense & Sparse vectors simultaneously).
- **Large Language Models (The Dual-Brain System):**
 - *The Cloud Brain (Speed):* Groq API (for rapid prototyping and real-time responses).
 - *The Isolated Brain (Security & Flex):* Qwen2.5-14B-Instruct (hosted on a Kaggle GPU backend to handle complex engineering logic and cross-lingual translation securely).

3. Infrastructure & Deployment Strategy

The infrastructure is designed to utilize available resources to their absolute maximum potential without creating system bottlenecks:

- **Database Node (Local):** Qdrant will run isolated inside a Docker container. The data volumes will be mounted exclusively to the solid-state drive to guarantee lightning-fast I/O operations and sub-second retrieval times, completely bypassing slower mechanical drives.
- **Orchestration Node (Local):** The Haystack Python environment and the lightweight BGE-m3 embedding model will run smoothly on the local quad-core CPU and memory footprint. This setup pairs perfectly with a multi-monitor workstation, allowing developers to dedicate separate screens to Docker logs, the Kaggle backend connection, and the main application code simultaneously.
- **AI Generation Node (Remote/API):** To prevent out-of-memory errors and avoid bottlenecking local graphics memory limits, all heavy text generation and query reformulation tasks are offloaded either to the Groq API or the Kaggle T4 GPUs.

4. Data Pipelines

The system is split into two primary pipelines to handle data flow securely and efficiently.

A. Indexing Pipeline (Data Ingestion)

1. **Ingestion:** Load the English mechanical maintenance manual.
2. **Parsing & Chunking:** Docling processes the document using hierarchical context. (e.g., A torque value is permanently tagged with its parent headers like "Engine → Cylinder Head").
3. **Embedding:** Text chunks are passed to the BGE-m3 model to generate two types of data:
 - *Dense Vectors:* To capture the overall semantic meaning.
 - *Sparse Vectors:* To capture exact keywords and precise numerical values.
4. **Storage:** Both vector types and the metadata are stored in the local Qdrant Docker container.

B. Query Pipeline (Retrieval & Generation)

1. **User Input:** The technician asks a question in Arabic.
2. **Query Rewriting:** The LLM intercepts the raw question and reformulates it into a highly optimized search query, extracting the exact mechanical terminology.
3. **Query Embedding:** The rewritten query is converted into Dense and Sparse vectors via BGE-m3 (utilizing its cross-lingual alignment).
4. **Hybrid Retrieval:** Qdrant combines semantic search with exact keyword/number matching (BM25) to pull the exact English manual sections.
5. **Professional Formatting:** The retrieved context and the rewritten query are injected into a strict PromptBuilder template. The LLM translates the mechanical data and outputs a perfectly structured, professional response (e.g., in bullet points or a JSON schema).

5. Technical Decisions & "The Why"

Why Hybrid Search?

Traditional semantic embeddings fail at differentiating precise numbers (e.g., 15 Nm vs. 55 Nm). Combining Dense search with Sparse (BM25) ensures we catch the exact numerical values with 100% accuracy.

Why Docling & Header-based Chunking?

Randomly splitting text destroys context. If the system retrieves "Torque to 30 Nm", the technician must know *which* bolt it refers to. Header-based chunking keeps the document's architectural "tree" attached to every single sentence.

Why BGE-m3?

It is currently the most efficient open-source model capable of crossing the Arabic-to-English language barrier while natively generating both Dense and Sparse vectors in a single pass, drastically simplifying the pipeline.

This document sets a solid engineering foundation for the MECHRABOT project.